

■ Clasificación Curricular Automática con MLP

*Evaluación Parcial N.º 1
Fundamentos de Deep Learning (DLY0100)*

Integrantes:

- Rahyzza Novoa
- Nicolás Salas

Profesora:

Elizabeth Chicata Castro

Índice

- 1 Problema y objetivo del modelo
- 2 Preparación de los datos
- 3 Arquitectura de la MLP
- 4 Funciones de activación, salida y pérdida
- 5 Épocas, learning rate y batch size
- 6 Efecto de la regularización
- 7 Comparación de configuraciones y modelo final
- 8 Resultados: accuracy, precision, recall y F1
- 9 Conclusiones y posibles mejoras

01

Problema y datos

Problema y objetivo del modelo

Problema. Clasificar automáticamente material educativo de matemáticas en una de cuatro categorías curriculares, a partir de 22 variables numéricas sobre su estructura, contenido y diseño.

Objetivo. Implementar una MLP en TensorFlow/Keras y justificar experimentalmente cada decisión de preprocesamiento, arquitectura, funciones, hiperparámetros y regularización.

100 mil

registros

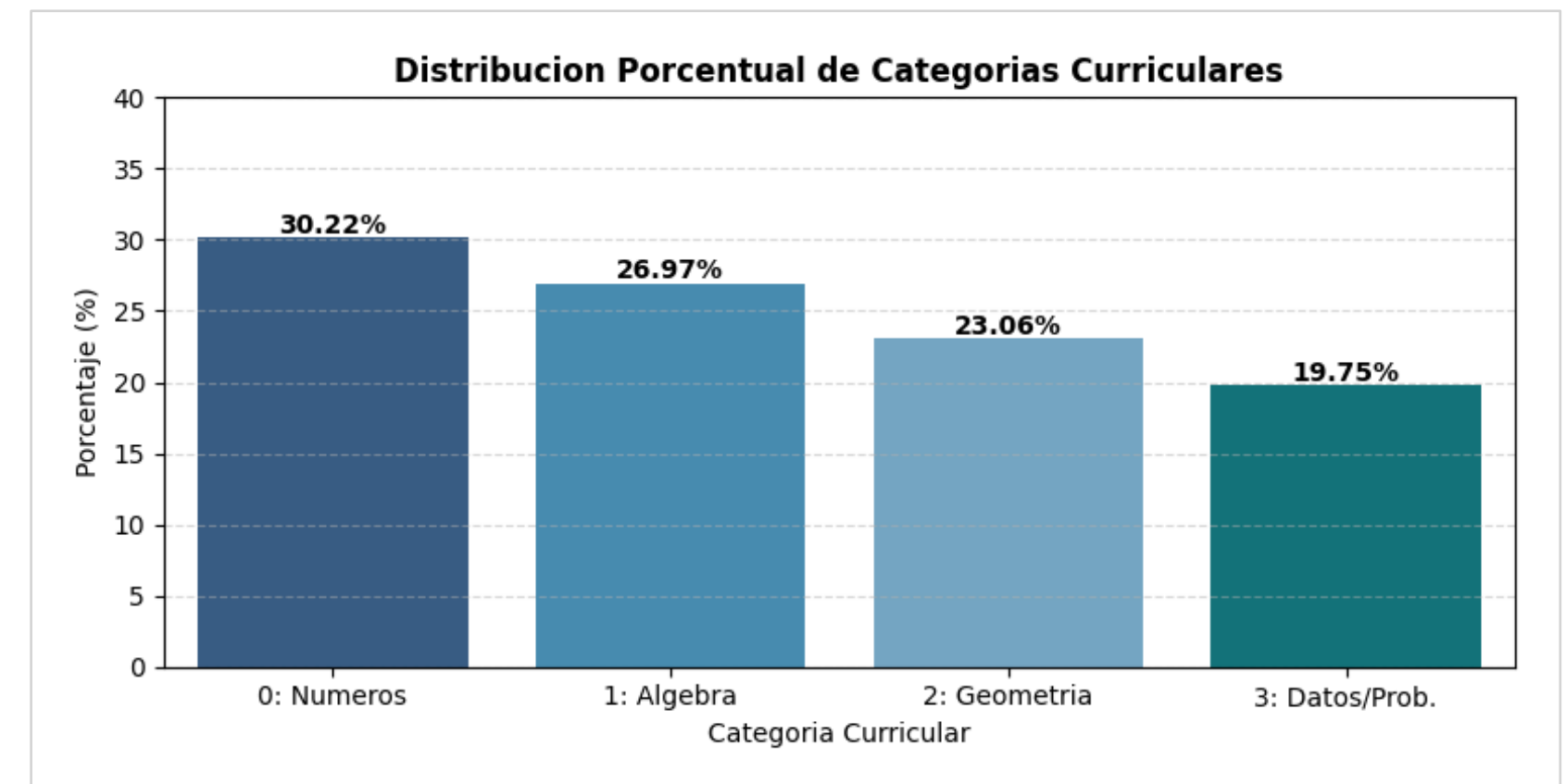
22

variables

4

categorías

Métrica principal: F1-score ponderado, por el desbalance moderado entre clases (ratio 1,53).



Preparación de los datos

1

Verificación

0 valores nulos y 0 filas duplicadas. El dataset llega limpio, por lo que no se requiere ETL.

2

Partición 70 / 15 / 15

Split estratificado con `random_state = 42`: 70.000 entrenamiento, 15.000 validación y 15.000 test, preservando la proporción de clases.

3

StandardScaler

Ajustado solo con train y aplicado a validación y test, lo que evita fuga de información. Deja todas las variables en media 0 y desviación 1.

Decisiones de reproducibilidad

Semilla fija (42) reiniciada antes de cada entrenamiento y una función común `entrenar_modelo()`: la única diferencia entre corridas es el parámetro que se evalúa. El conjunto de test no se usa hasta la evaluación final.



02

*Modelo y
experimentos*

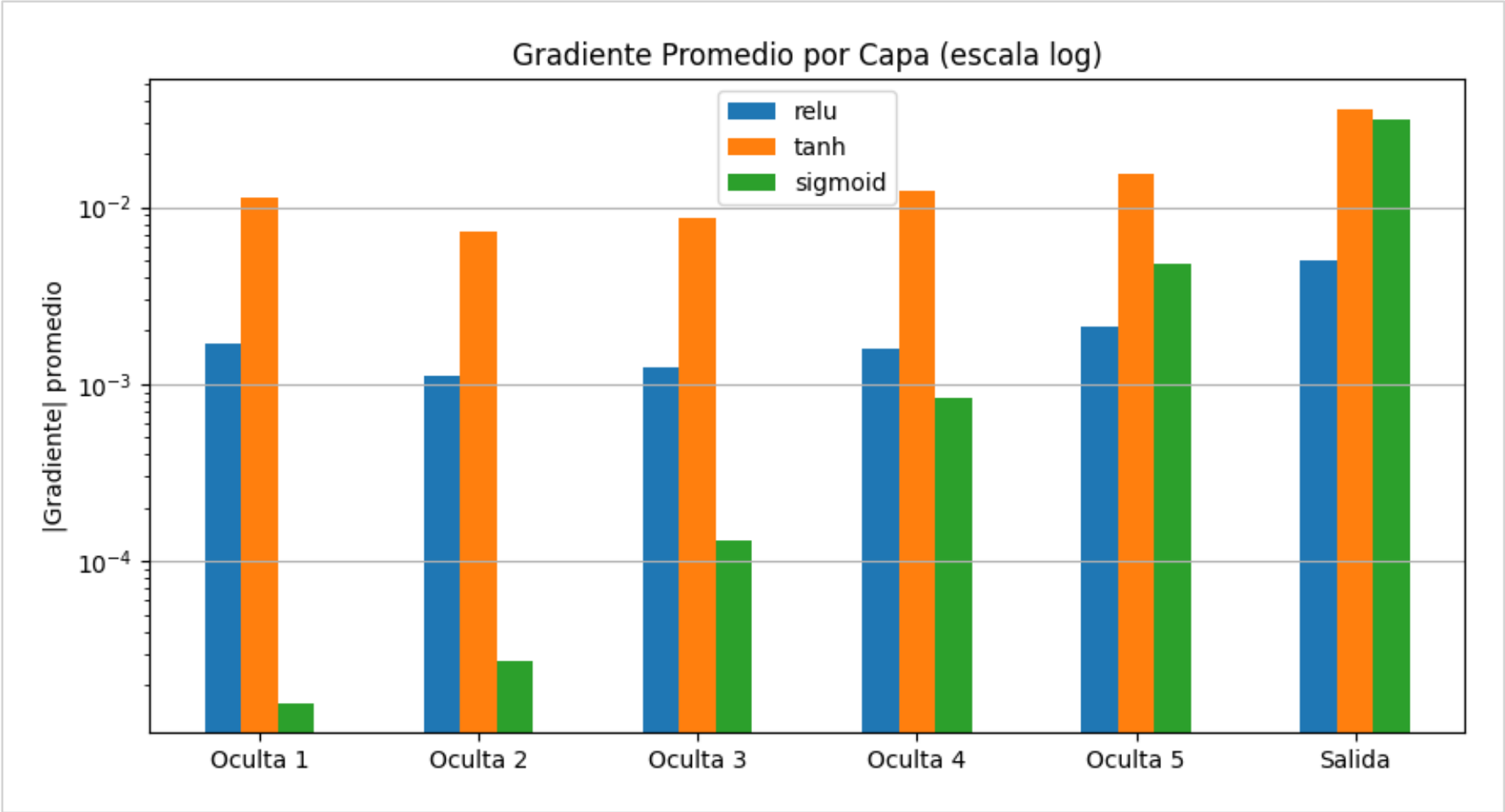
Arquitectura de la MLP



3.684 parámetros entrenables

Pérdida Sparse Categorical Cross-Entropy	Optimizador Adam lr = 0.0005	Batch size 256	Épocas Early Stopping patience = 10
---	---	-------------------------------------	--

Funciones de activación



Gradiante medio por capa en una red de 5 capas, antes de entrenar

1.959×

Sigmoide

Atenuación del gradiente entre la salida y la primera capa

2,9×

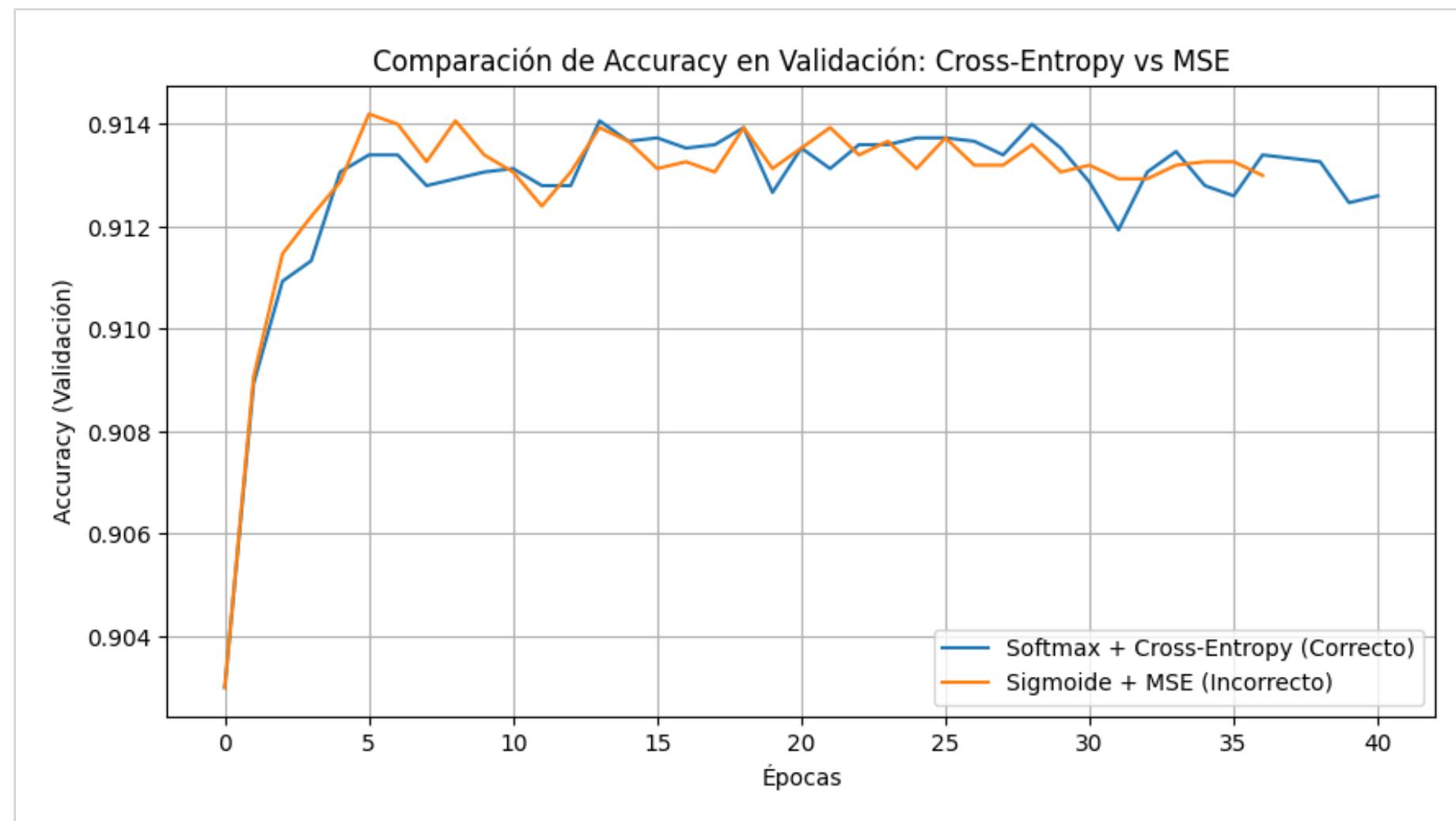
ReLU

Atenuación entre la salida y la primera capa

Activación	F1 (64-32)	Épocas red profunda
ReLU	0,9128	36
LeakyReLU	0,9130	—
Tanh	0,9146	88
Sigmoide	0,9152	56

Selección: ReLU. En 2 capas todas rinden igual ($\Delta F1 = 0,0024$), pero ReLU no desvanece el gradiente y converge antes al profundizar.

Función de salida y función de pérdida



Se compara val_accuracy: las pérdidas CE y MSE tienen escalas distintas

Softmax + Cross-Entropy

F1 = 0,9128

Las 4 salidas suman exactamente 1,000 en todas las muestras: distribución de probabilidad válida.

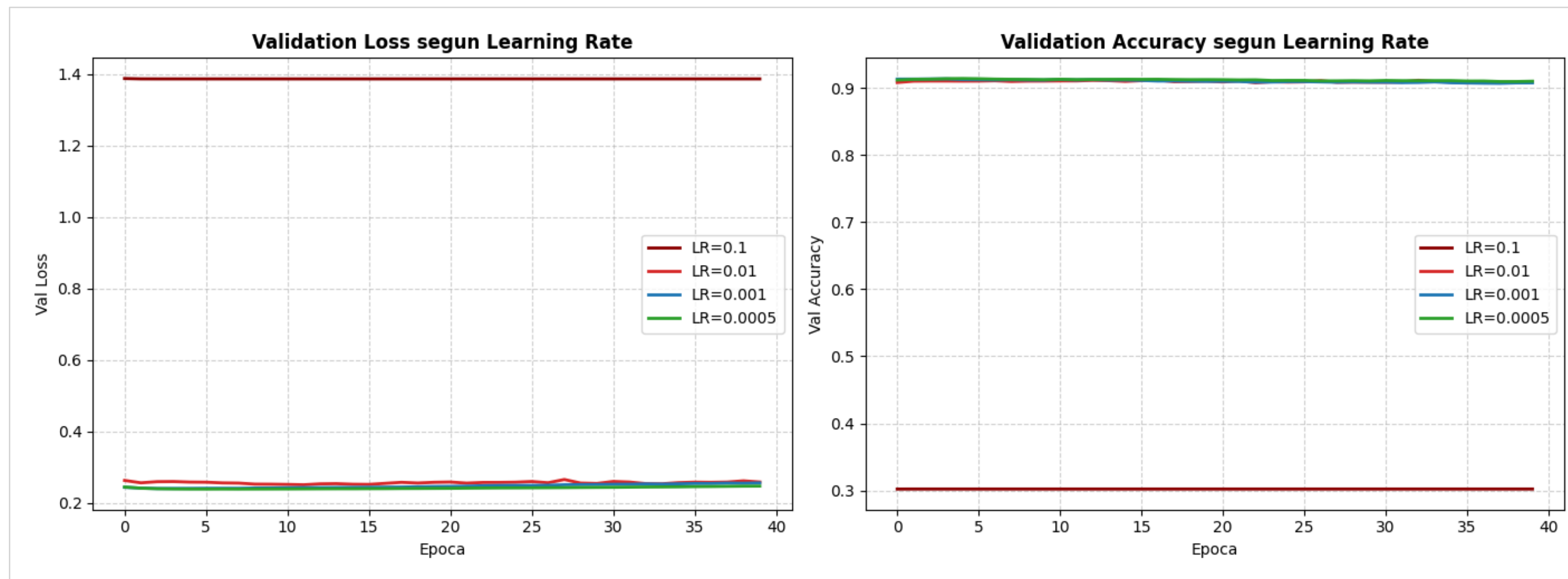
Sigmoide + MSE

F1 = 0,9132

La suma de las salidas varía entre 0,82 y 1,49: no son probabilidades interpretables.

Selección: Softmax + Sparse Categorical Cross-Entropy, por coherencia con 4 clases mutuamente excluyentes, no por accuracy.

Learning rate

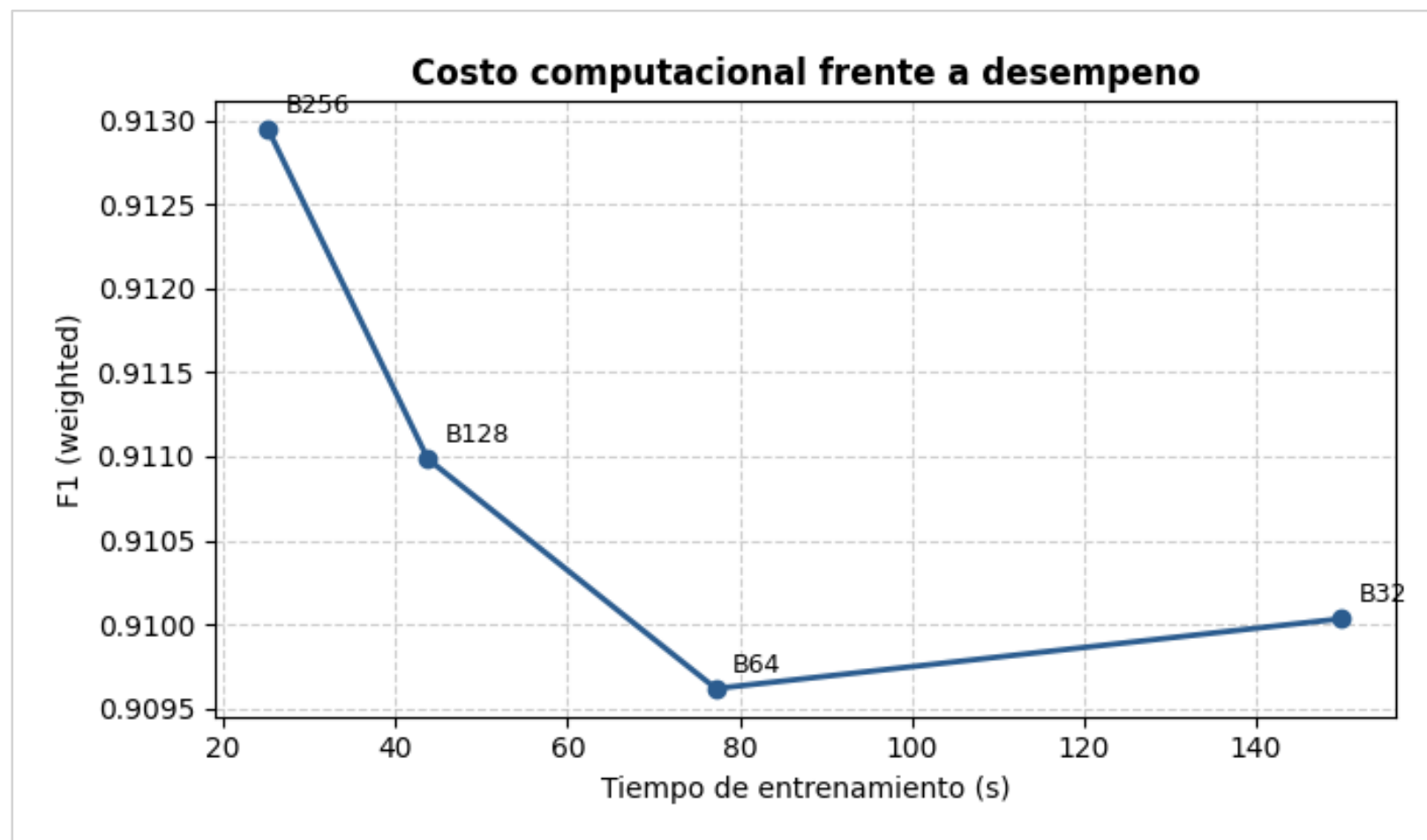


$\alpha = 0.1$ colapsa. Accuracy 30,22% (clase mayoritaria), val_loss $\approx \ln 4$.

0.01 a 0.0005: Δ F1 de solo 0,0024. Adam amortigua la sensibilidad a la tasa.

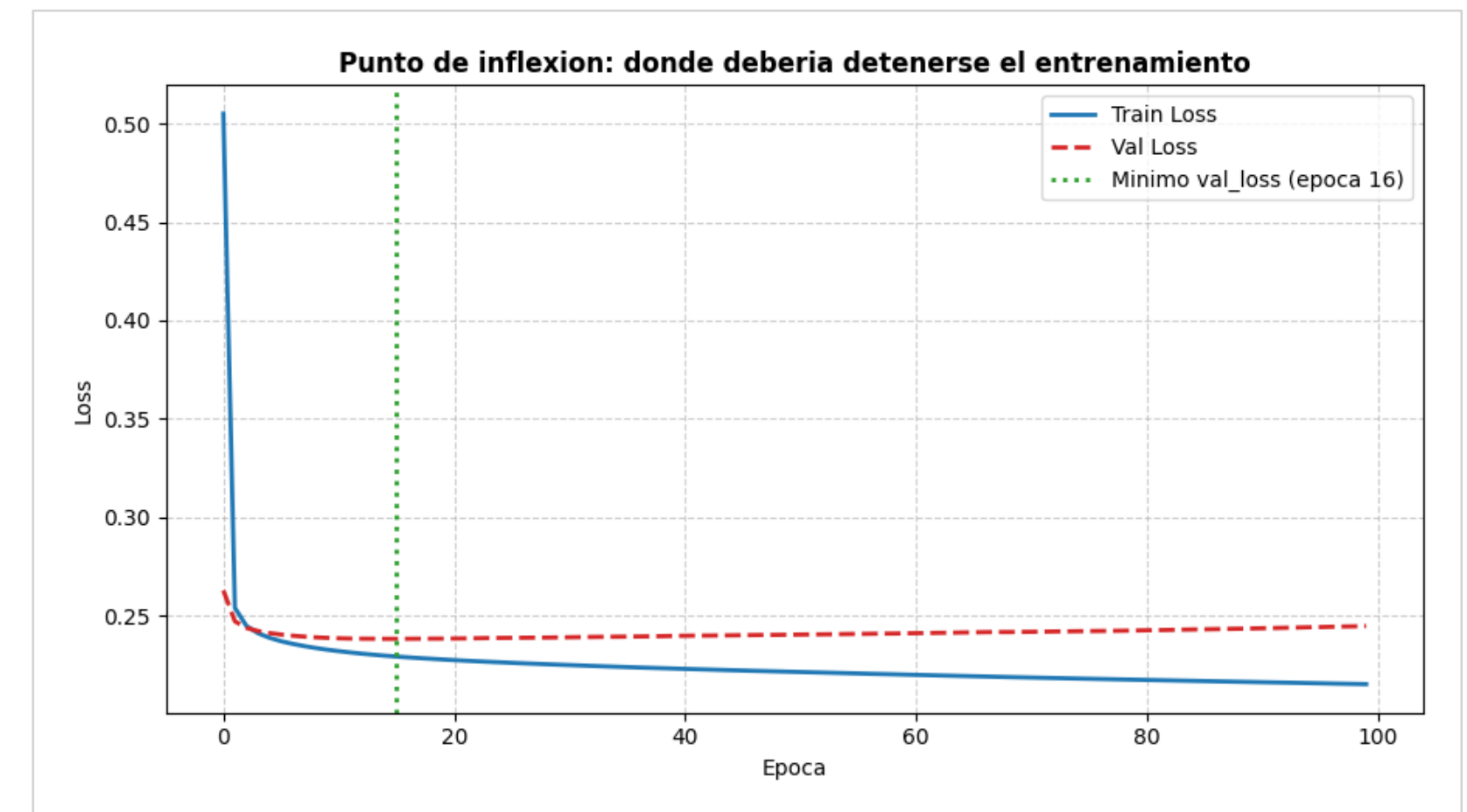
Elegido 0.0005: F1 0,9100 y menor val_loss (0,2394).

Batch size y épocas



Batch 256

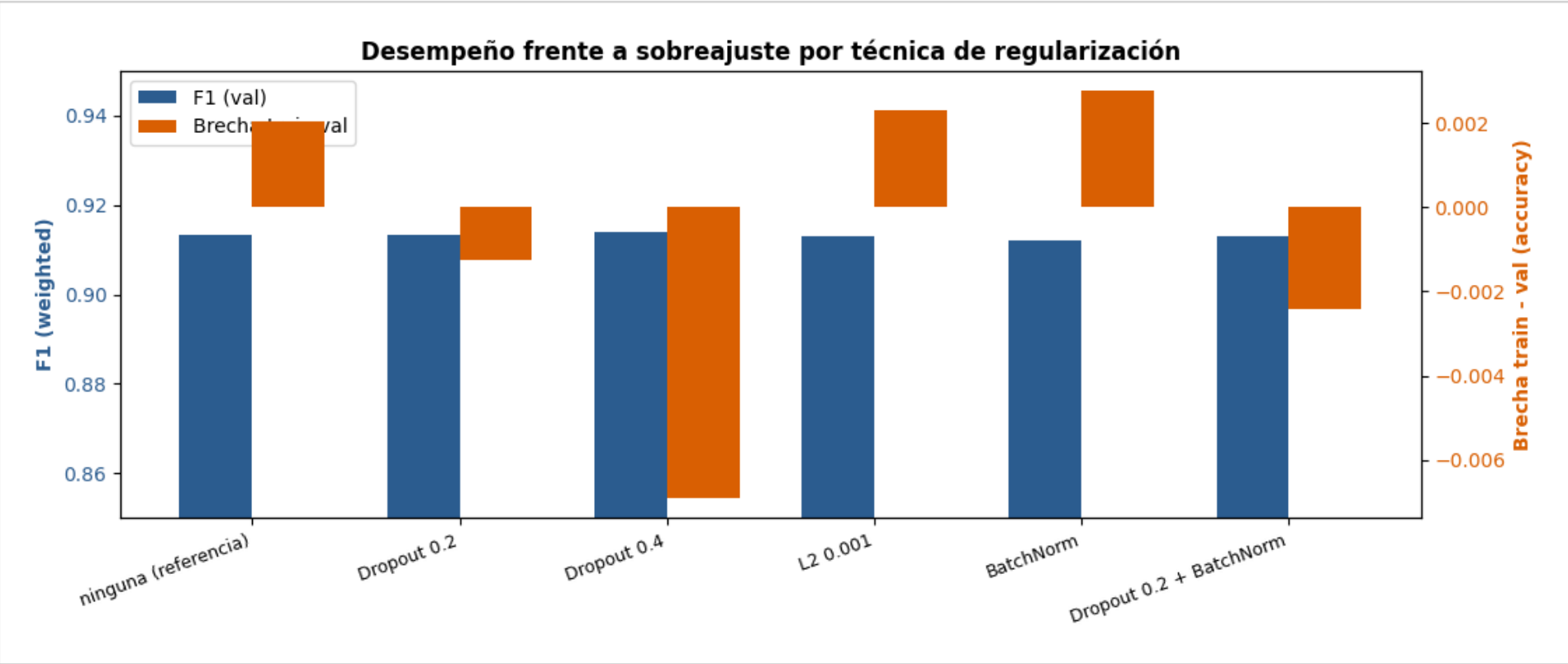
F1 0,9129 en 25 s, frente a 0,9100 en 150 s con batch 32: cerca de 6 veces más rápido y con mejor desempeño.



Early Stopping

val_loss toca su mínimo en la época 16; las 84 restantes no mejoran. Se usa patience = 10 con restore_best_weights.

Efecto de la regularización



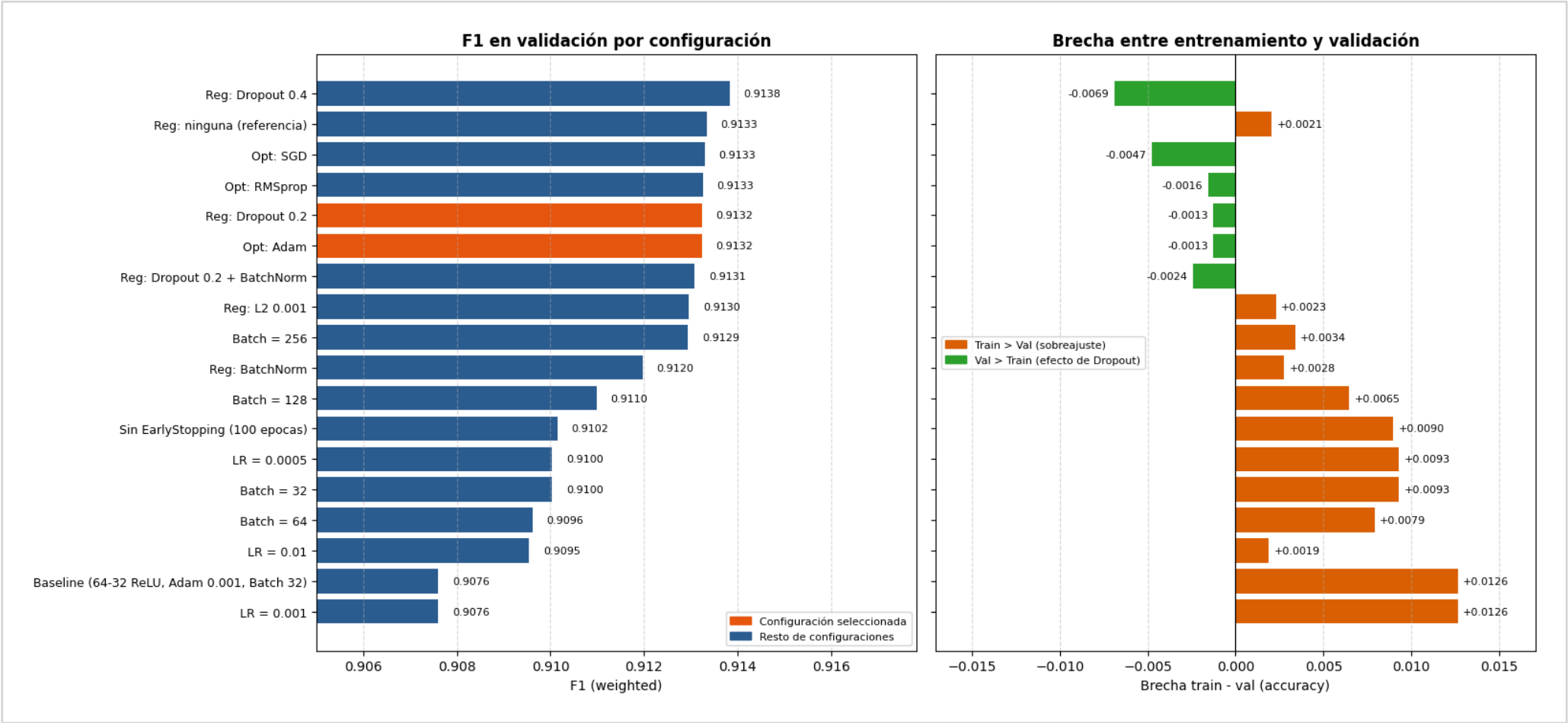
F1 (barras azules) frente a brecha train-val (barras naranjas)

Técnica	F1	val_loss mín.	Brecha
Ninguna	0,9133	0,2381	+0,0021
Dropout 0.2	0,9132	0,2341	-0,0013
Dropout 0.4	0,9138	0,2353	-0,0069
L2 0.001	0,9130	0,2459	+0,0023
BatchNorm	0,9120	0,2423	+0,0028
Dropout + BN	0,9131	0,2355	-0,0024

Hallazgo principal

Las 6 variantes quedan en un rango de F1 de solo 0,0019, porque el sobreajuste de base ya era leve. Se elige Dropout 0.2: su diferencia con Dropout 0.4 (0,0006) está bajo el umbral de 0,005, y tiene el menor val_loss (probabilidades mejor calibradas) y la menor brecha absoluta. Las brechas negativas se deben a que la accuracy de entrenamiento se mide con neuronas apagadas.

Comparación de configuraciones



Las 19 corridas del notebook principal; en naranja, la configuración seleccionada

Contribución de cada ajuste

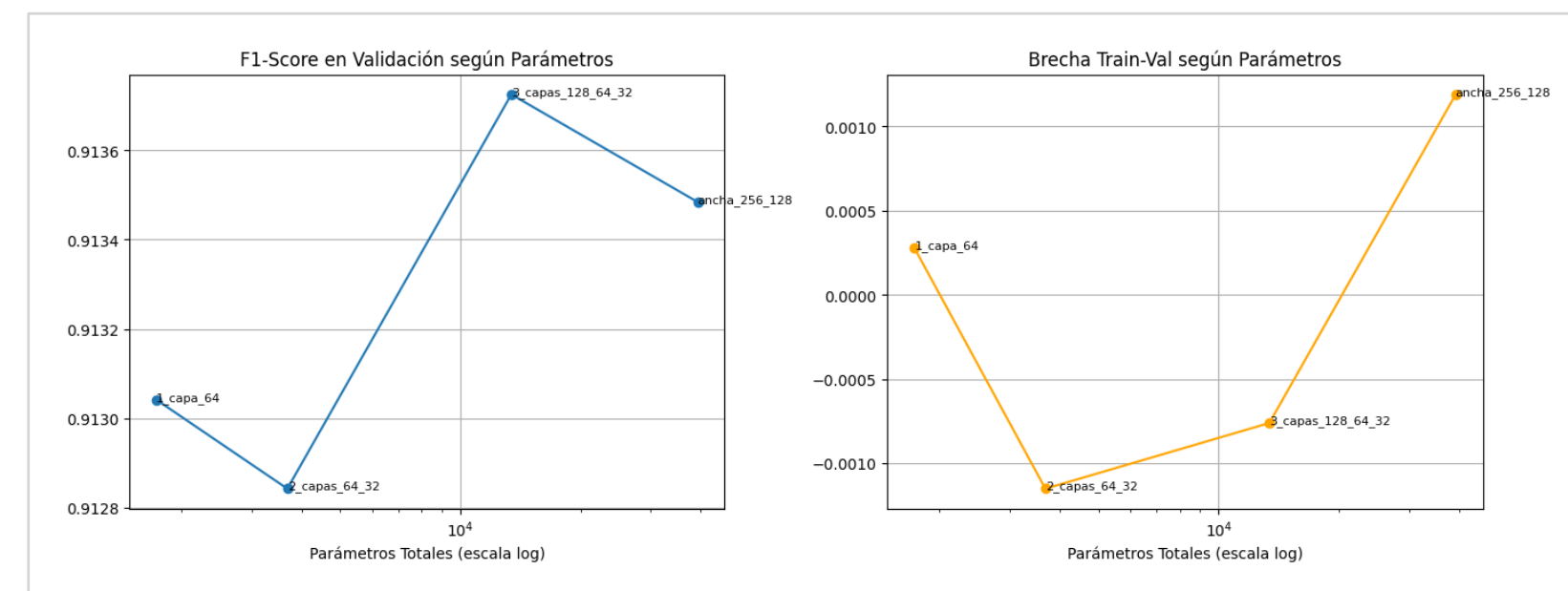
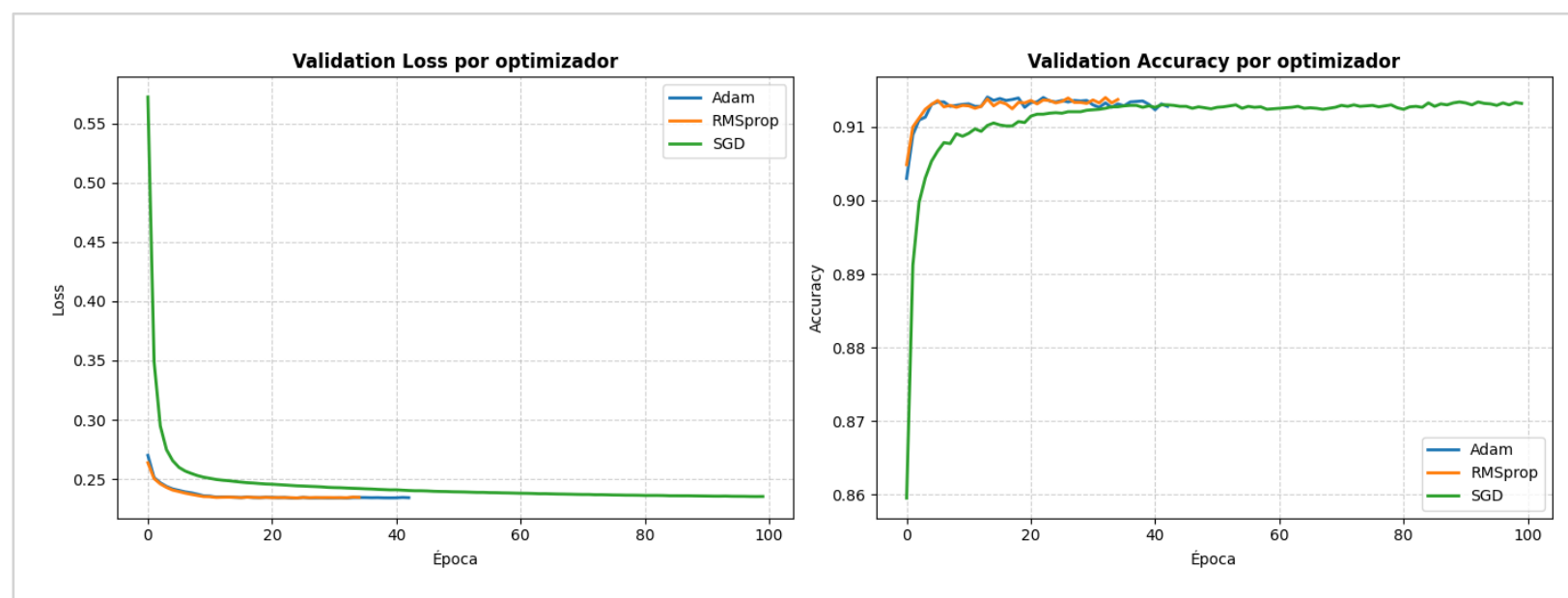
Etap	F1	Δ F1
Baseline	0,9076	—
LR 0.0005	0,9100	+0,0024
Batch 256	0,9129	+0,0029
Early Stopping	0,9133	+0,0004
Dropout 0.2	0,9132	−0,0001

Criterio: F1 ponderado; ante diferencias menores a 0,005, menor val_loss y menor brecha; el costo solo como desempate.

Lectura

El learning rate y el batch size explican +0,0053 de los +0,0056 puntos de mejora. La regularización no sube el F1, pero mejora la calibración y la generalización.

Optimizadores y arquitecturas



Optimizadores (B5)

Adam 0,9132 · RMSprop 0,9133 · SGD 0,9133

RMSprop converge un 24,5% más rápido; SGD no alcanza a converger en 100 épocas.

Se mantiene Adam: el learning rate, el batch y la regularización se calibraron con él.

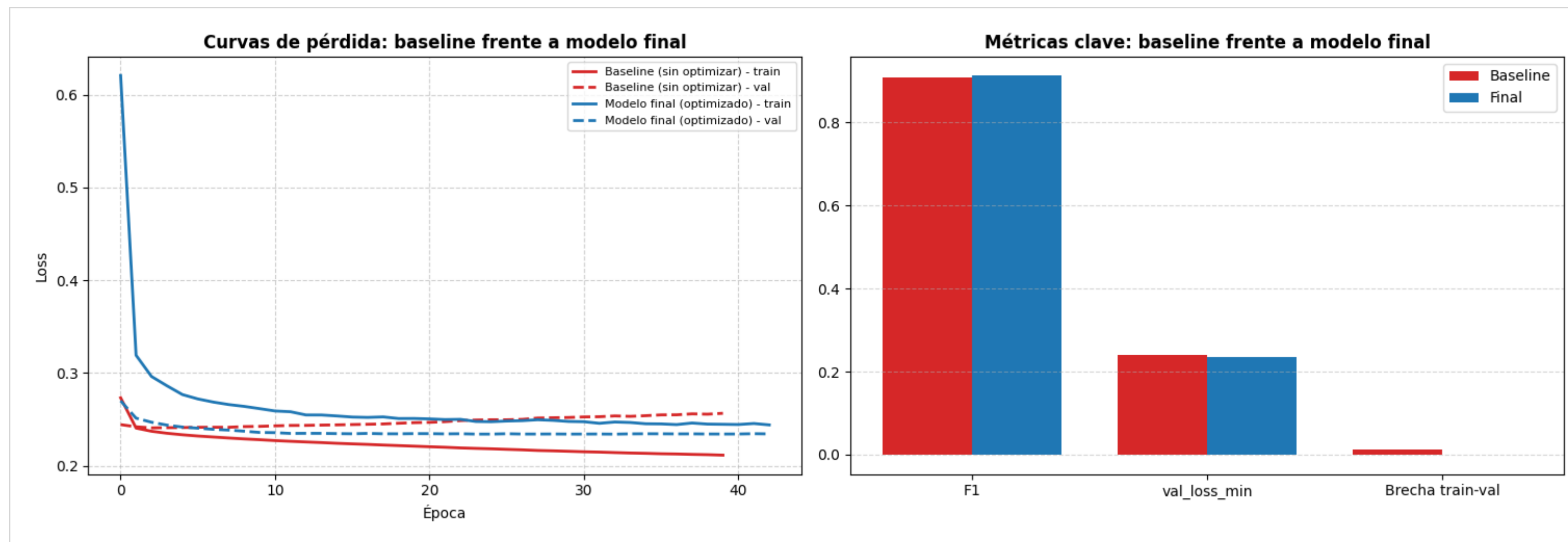
Arquitecturas (A3)

64 · 64-32 · 128-64-32 · 256-128

Con 22 veces más parámetros, el F1 varía solo 0,0009 y la brecha se mantiene mínima.

Se mantiene 64-32: ninguna la supera en más de 0,005 y es la base de la calibración.

Modelo final frente al baseline



+0,0056

F1 validación

0,9076 → 0,9132

-2,8%

val_loss mínima

0,2409 → 0,2341

-0,0013

Brecha train-val

desde +0,0126

≈ 5×

Más rápido

152 s → 31 s



03

Resultados y conclusiones

Resultados en test

0,9108

Accuracy

0,9108

Precision (weighted)

0,9108

Recall (weighted)

0,9108

F1-score (weighted)

Categoría	Precision	Recall	F1	Soporte
Números	0,9094	0,9007	0,9050	4.533
Álgebra	0,9091	0,9150	0,9120	4.045
Geometría	0,9120	0,9078	0,9099	3.460
Datos y Prob.	0,9139	0,9240	0,9189	2.962

Validación 0,9132 → test 0,9108: diferencia de solo 0,0025. Las decisiones tomadas sobre validación no se sobreajustaron a ese conjunto.

Cálculo con la matriz real: Números

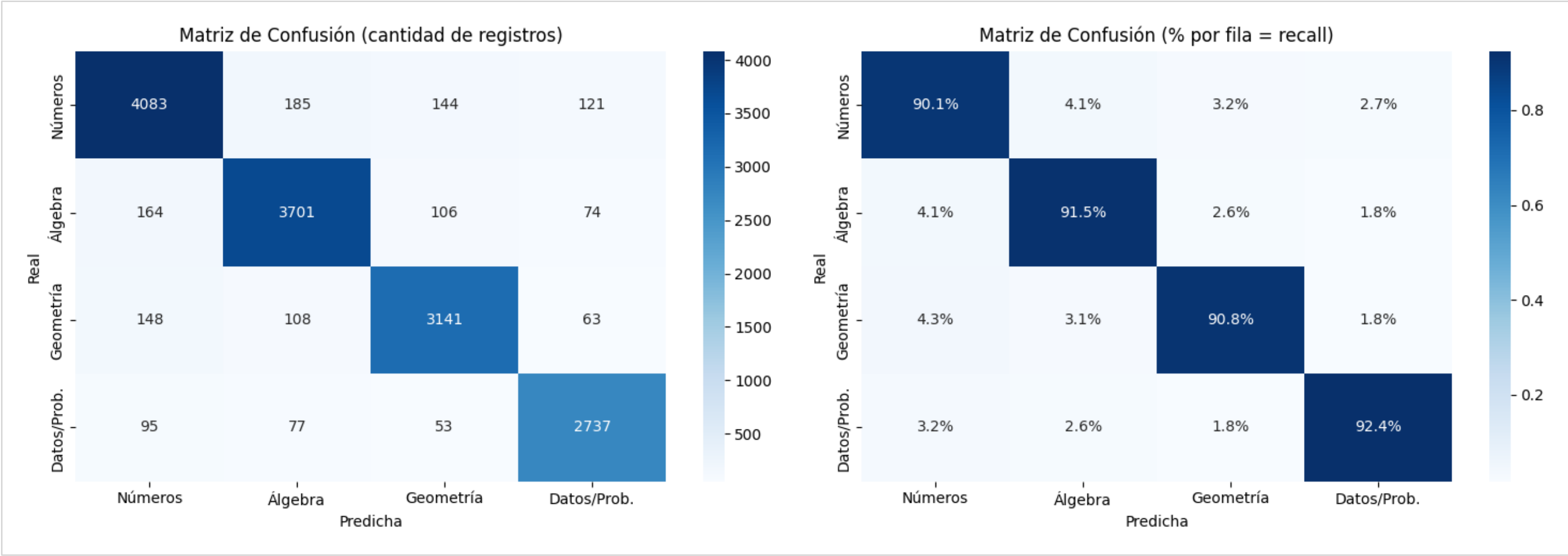
Precision = $VP / (VP + FP)$
= $4.083 / (4.083 + 407) = 0,9094$

Recall = $VP / (VP + FN)$
= $4.083 / (4.083 + 450) = 0,9007$

F1 = $2 \cdot P \cdot R / (P + R) = 0,9050$

El F1 ponderado calculado a mano coincide con sklearn: 0,9108.

Matriz de confusión



185

Números → Álgebra

Confusión más frecuente

164

Álgebra → Números

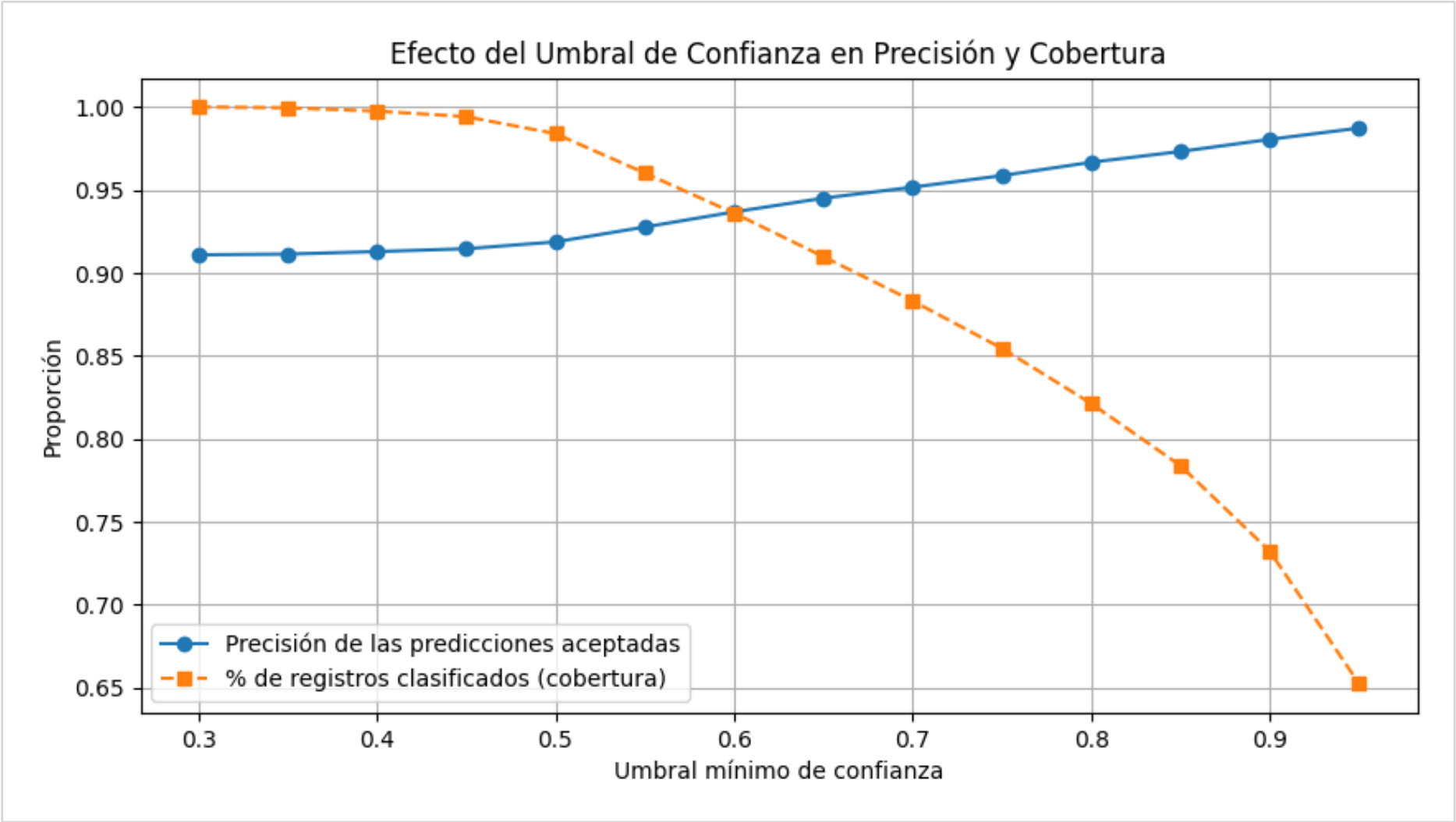
La misma pareja, en sentido inverso

148

Geometría → Números

Números aparece en las tres

Cómo mejorar precisión y recall



Umbral	Precisión	Cobertura
Sin umbral	0,9108	100%
0,60	0,9367	93,6%
0,70	0,9516	88,3%
0,80	0,9666	82,1%
0,90	0,9804	73,2%

Propuestas

- Umbral $\geq 0,80$: precisión de 96,7%; el 17,9% restante pasa a revisión manual.
- `class_weight` para subir el recall de Números (no para la clase minoritaria, que ya es la mejor).
- Nuevas variables que distingan Números de Álgebra.

Conclusiones y posibles mejoras

Conclusiones

- MLP 64-32 con ReLU, Softmax y Cross-Entropy; Adam 0.0005, batch 256, Dropout 0.2 y Early Stopping. F1 en test: 0,9108.
- La mayor mejora proviene del learning rate y del batch size; la regularización mejora la calibración más que el F1.
- Las funciones y arquitecturas rinden de forma equivalente en esta red; las diferencias aparecen en el gradiente y en la velocidad de convergencia.
- El desempeño en test confirma la generalización: solo 0,0025 bajo validación.

Posibles mejoras

- Búsqueda conjunta de hiperparámetros (Keras Tuner), en lugar de variar uno a la vez.
- Validación cruzada estratificada para estimar la variabilidad de las métricas.
- Umbral de confianza con revisión manual para los registros de baja certeza.
- Nuevas variables para separar Números de Álgebra, y `class_weight` para subir el recall de Números.



■ Gracias

¿Preguntas?

Repositorio

github.com/Nicolas-Salas/deep-learning-ep1